Project: VVargetable

Project Statement

This game is based on a famous Flash game named “FLEAG VS. MUTT” developed by gameturner.com, and we modified it to have two game modes and the special skills of both characters to be different and be reused through cooldown.

1.1 Overview

1.1.1 Rule

Each player takes turns throwing vegetables to reduce each other’s health points, or HP and using various skills to help win the game.

The game ends when one’s health points run out. The player who lets the health points run out loses.

1.1.2 Example

Fig 1. Example

Check the wind direction and speed, then press and hold the character to increase the throwing power gauge to the desired level. You can use various skills by clicking to defend or enhance your attack strategy.

1.1.3 Scenes

Fig 2. Start screen

Upon starting up the game, you will be greeted by the starting screen; click play to see the main screen.

Fig 3. Main screen

Click "How to Play" to see the how-to-play screen, and click "Play" to see the Mode-Selection screen.

Fig 4. How-to screen

This screen introduces the rules of the game. Click "Play" to see the Mode-Selection screen.

Fig 5. Select-mode screen

There are two game modes, differing in background, attack damage, and skill cooldown.

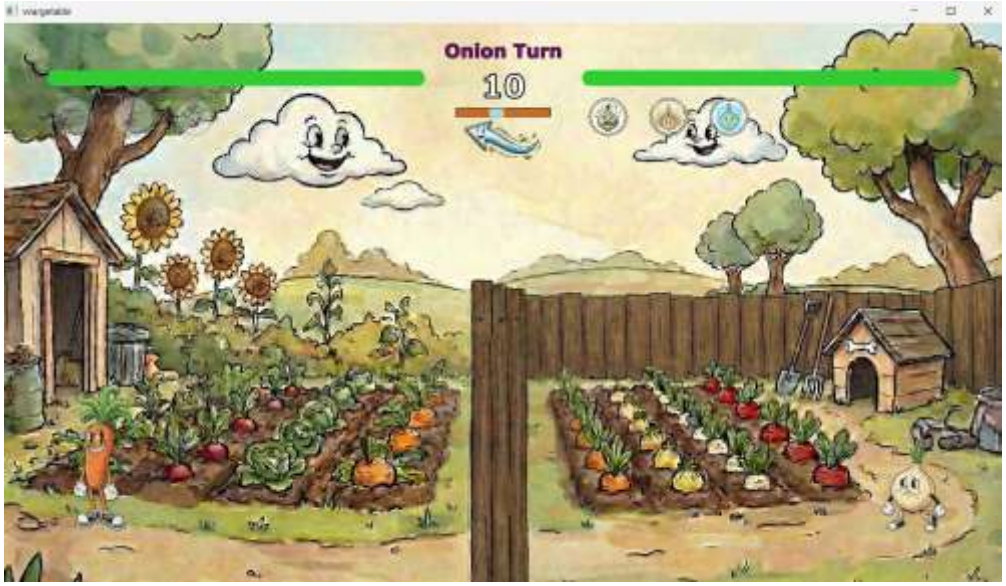
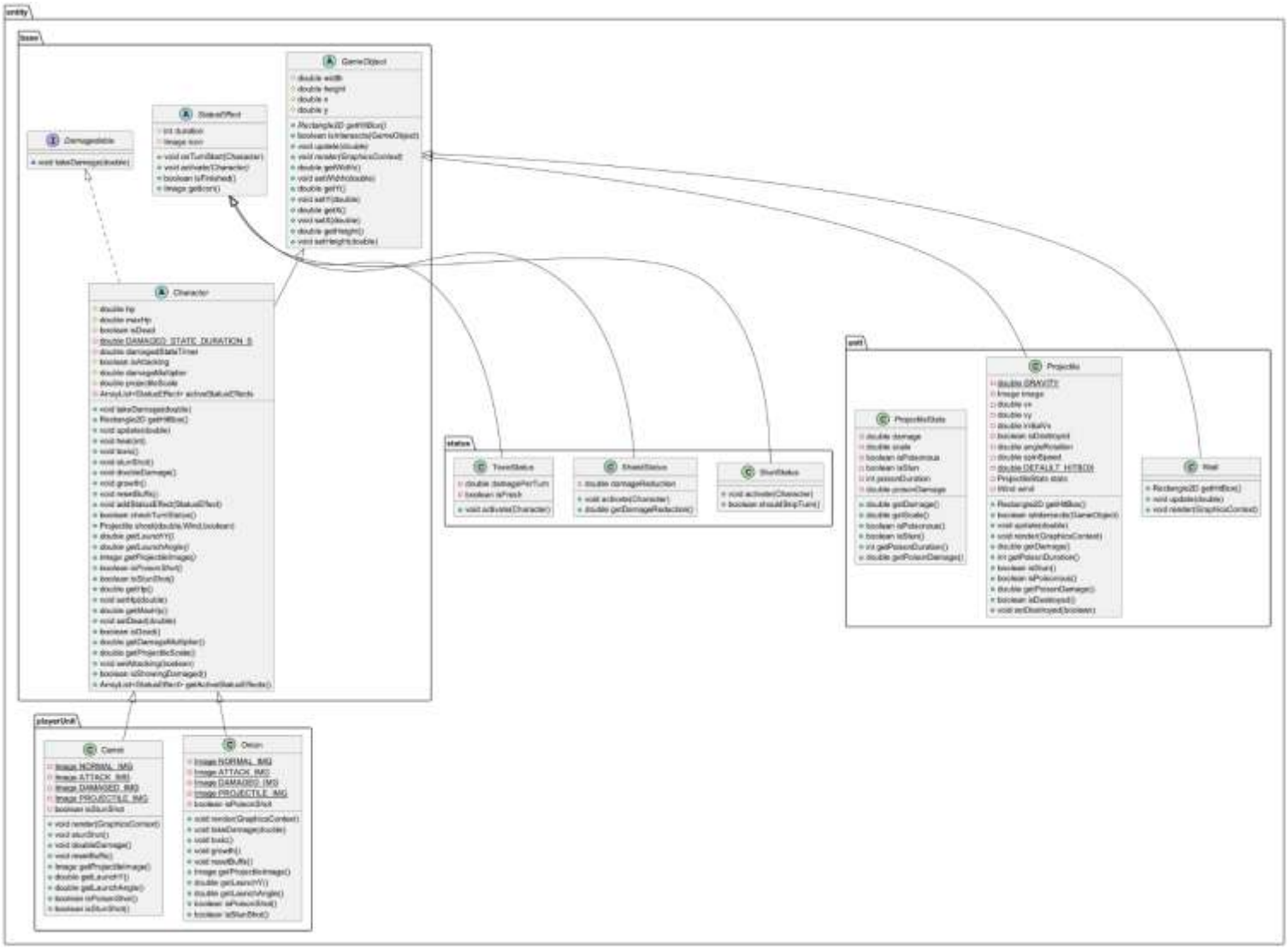


Fig 6. Classic mode

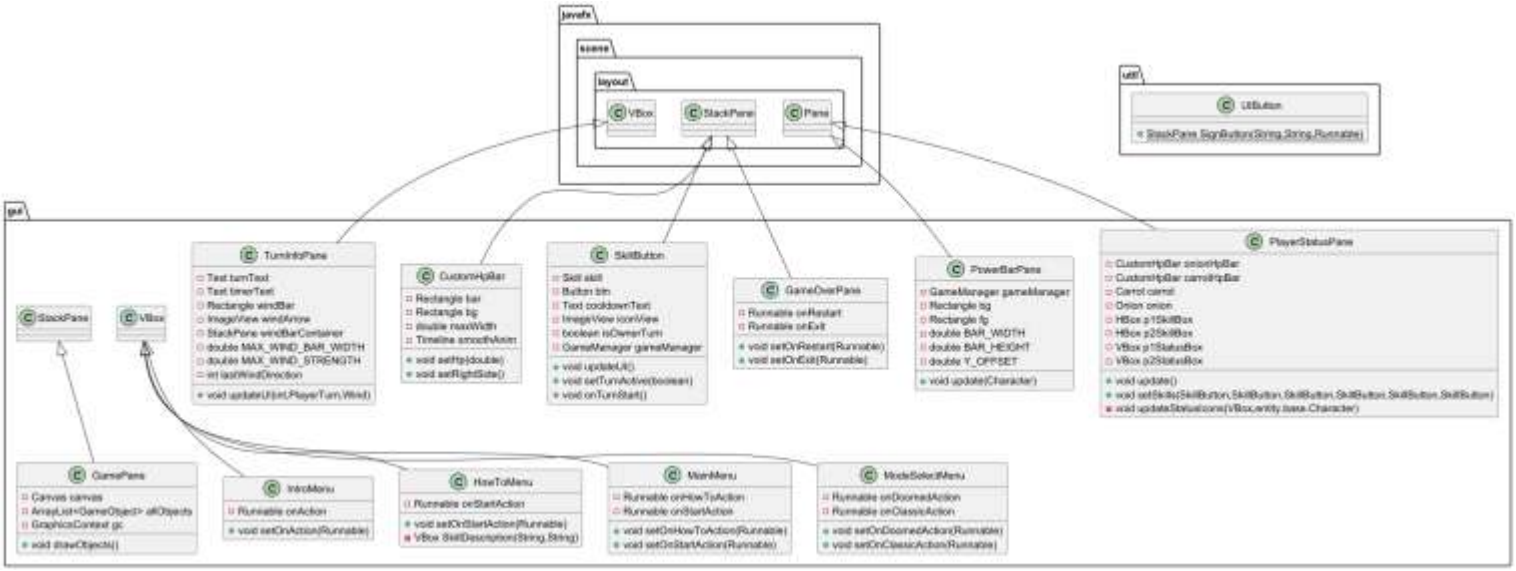
Fig 7. Doomed mode

1.1.4 Class Diagram

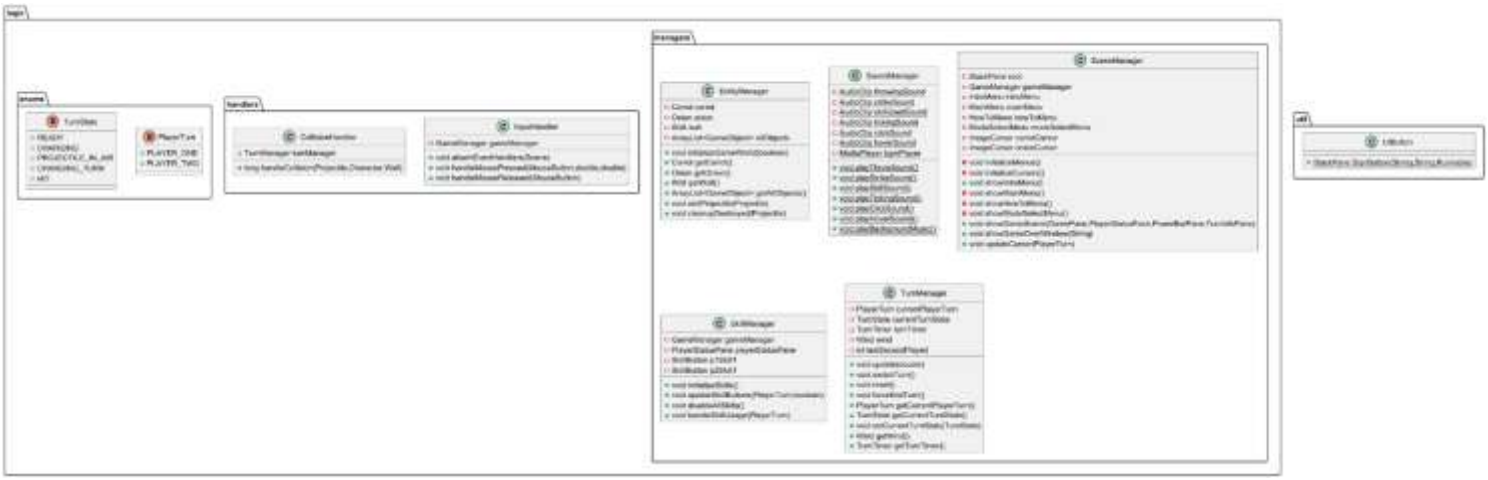
package entity



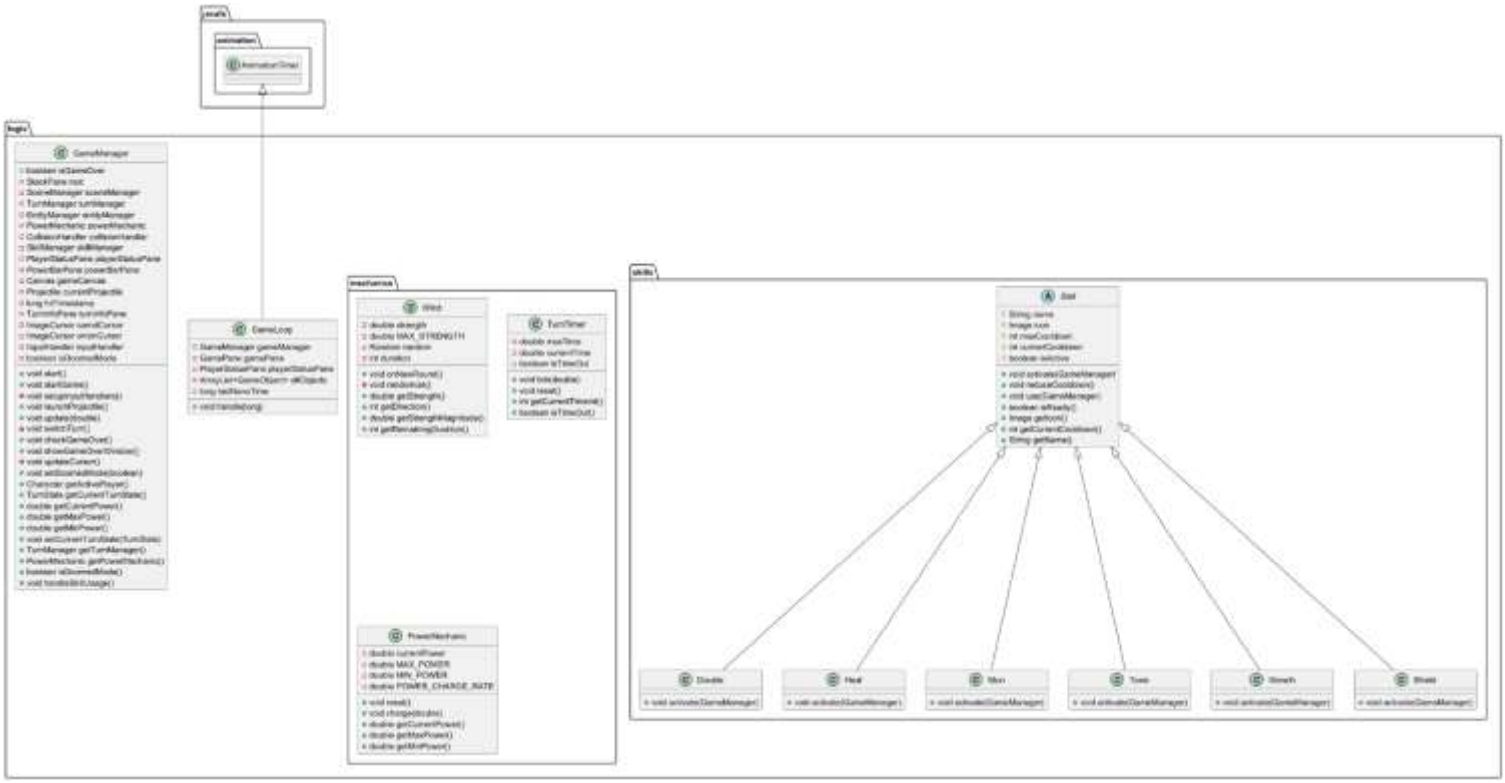
package gui and util



package logic (.enums, .handlers, .manager)



package logic (.mechanics, .skills, GameManager, GameLoop)



2. Details

2.1 package entity

2.1.1 package base

2.1.1.1 Abstract Class GameObject

This abstract class represents the fundamental base for all objects in the game world, managing position, dimensions, collision detection, and rendering requirements.

2.1.1.1.1 Fields

# double width	The width of the game object.
# double height	The height of the game object.
# double x	The position of the object on the X-axis.
# double y	The position of the object on the Y-axis.

2.1.1.1.2 Constructor

+ GameObject(double x, double y)	Initializes the object by setting the x and y coordinates to the provided values.

2.1.1.1.3 Methods

+ boolean isIntersects(GameObject other)	- Checks for collision between this object and another GameObject. - Returns true if this.getHitBox() intersects with other.getHitBox().
+ *Rectangle2D getHitBox()*	Abstract method to return the specific bounding box used for collision detection.
+ *void update(double deltaTime)*	Abstract method to update the object's logic based on the elapsed time deltaTime.
+ *void render(GraphicsContext gc)*	Abstract method to draw the object onto the screen using the provided GraphicsContext.
+ getter/setter	Standard getters and setters for x, y, width, and height.

2.1.1.2 Interface Damagedable

This interface defines the behavior for any game entity that is capable of receiving damage.

2.1.1.2.1 Method

void takeDamage(double amount)	Defines the action to be taken when the object receives a specified amount of damage.

2.1.1.3 Class Character extends GameObject implements Damagedable

This abstract class represents the base entity for all playable characters in the game. It manages health, status effects, shooting mechanics, and rendering states.

2.1.1.3.1 Fields

# double hp	Current health of the character.
# double maxHp	Maximum health of the character.

# boolean isDead	Flag indicating if the character is dead.
# boolean isAttacking	Flag indicating if the character is currently attacking.
# double damageMultiplier	Multiplier applied to projectile damage (default 1.0).
# double projectileScale	Scale factor for the projectile size (default 1.0).
- double damagedStateTimer	Timer to track how long the character stays in the "damaged" visual state.
- static final double DAMAGED_STATE_DURATION_S	Constant defining the duration of the damaged state (1.0 second).
- final ArrayList<StatusEffect> activeStatusEffects	List of status effects currently applied to the character.

2.1.1.3.2 Constructor

+ Character(double x, double y, int health)	- Calls the superclass constructor GameObject(x, y). - Sets both hp and maxHp to the provided health.

2.1.1.3.3 Methods

+ void takeDamage(double amount)	- Reduces hp by the specified amount using setHp. - Updates death status using setDead. - Sets damagedStateTimer to DAMAGED_STATE_DURATION_S. - Plays a strike sound via SoundManager.
+ Rectangle2D getHitBox()	Returns a specific Rectangle2D representing the character's hitbox, offset from its position.
+ void update(double deltaTime)	Updates logic per frame. Specifically, it decreases damagedStateTimer by deltaTime if it is greater than 0.
+ void heal(int amount)	Increases hp by the specified amount using setHp.

+ void resetBuffs()	Resets temporary buffs: sets damageMultiplier and projectileScale back to 1.0.
+ void addStatusEffect(StatusEffect effect)	Adds the specified effect to activeStatusEffects.
+ boolean checkTurnStatus()	- Iterates through activeStatusEffects. - If an effect is a StunStatus, checks if the turn should be skipped. - Calls effect.onTurnStart(this) for active effects. - Removes effects that are finished. - Returns true if the turn should be skipped, false otherwise.
+ Projectile shoot(double power, Wind wind, boolean isDoomedMode)	- Calculates totalDamage based on power, game mode factor, and damageMultiplier. - Creates ProjectileStats using the calculated damage and scale. - Calculates the spawn position (startX) to center the projectile relative to the character. - Instantiates and returns a new Projectile. - Calls resetBuffs() after shooting.
+ void setHp(double hp)	Sets the hp value, ensuring it stays between 0 and maxHp.
+ void setDead(double hp)	Sets isDead to true if hp is less than or equal to 0.
Abstract Methods	getLaunchX(), getLaunchY(), getLaunchAngle(), getProjectileImage(), isPoisonShot(), isStunShot().
Other Getters/Setters	
Empty Methods	toxic(), stunShot(), doubleDamage(), growth() (Placeholder methods for potential subclass overrides).

2.1.1.4 Abstract Class StatusEffect

This abstract class represents a base for temporary effects that can be applied to characters, such as buffs or debuffs. It manages the duration and activation logic per turn.

2.1.1.1.1 Fields

# int duration	The number of turns the effect will remain active.
# Image icon	The icon image representing the status effect.

2.1.1.1.2 Constructor

+ StatusEffect(int duration, String iconPath)	- Assigns the duration field. - Attempts to load the icon using the provided iconPath. - If loading fails, catches the exception, prints an error message, and sets icon to null.

2.1.1.1.3 Methods

+ void onTurnStart(Character target)	If duration > 0: - Calls the abstract activate(target) method to apply the specific effect logic. - Decrements duration by 1.
+ *void activate(Character target)*	Abstract method defining the specific behavior of the status effect when it activates on a target.
+ boolean isFinished()	Returns true if the duration is less than or equal to 0, indicating the effect has expired.
+ Image getIcon()	Getter method for the status effect's icon.

2.1.2 package playerUnit

2.1.2.1 Class Carrot extends Character

This class represents the Carrot player unit. It handles specific rendering logic, projectile launch positions, and unique skill activations for the Carrot character.

2.1.2.1.1 Fields

- static Image NORMAL_IMG, ATTACK_IMG, DAMAGED_IMG, PROJECTILE_IMG	Static resources for the different states of the character and its projectile.
- boolean isStunShot	Flag indicating if the next shot will apply a stun effect.

2.1.2.1.2 Constructor

+ Carrot(double x, double y, int health)	- Calls the superclass constructor super(x, y, health). - Sets the width and height to 1/5th of the NORMAL_IMG dimensions.

2.1.2.1.3 Methods

+ void render(GraphicsContext gc)	- If isShowingDamaged() is true, use DAMAGED_IMG. - If isAttacking is true, use ATTACK_IMG. - Otherwise, use NORMAL_IMG. - Draws the selected image at (x, y).
+ void stunShot()	Sets isStunShot to true.
+ void doubleDamage()	Sets the inherited damageMultiplier to 2.0.
+ void resetBuffs()	Calls super.resetBuffs() to reset standard multipliers and sets isStunShot to false.
+ double getLaunchX()	Returns the starting X coordinate for projectiles (getX() + 50).
+ double getLaunchY()	Returns the starting Y coordinate for projectiles, centered vertically relative to the projectile's scale.
+ double getLaunchAngle()	Returns the fixed launch angle of -60 degrees.
+ boolean isPoisonShot()	Returns false (Carrot does not use poison).
+ boolean isStunShot()	Returns the value of isStunShot.
+ Image getProjectileImage()	Returns PROJECTILE_IMG.

2.1.2.1 **Class Onion** extends Character

This class represents the Onion player unit. It handles specific rendering logic, projectile launch positions, and unique skill activations for the Carrot character.

2.1.2.1.1 Fields

- static Image NORMAL_IMG, ATTACK_IMG, DAMAGED_IMG, PROJECTILE_IMG	Static resources for the different states of the character and its projectile.
- boolean isPoisonShot	Flag indicating if the next shot will apply a poison effect.

2.1.2.1.2 Constructor

+ Onion(double x, double y, int health)	- Calls the superclass constructor super(x, y, health). - Sets the width and height to 1/5th of the NORMAL_IMG dimensions.

2.1.2.1.3 Methods

+ void render(GraphicsContext gc)	- If isShowingDamaged() is true, use DAMAGED_IMG. - If isAttacking is true, use ATTACK_IMG. - Otherwise, use NORMAL_IMG. - Draws the selected image at (x, y).
+ void takeDamage(double amount)	- Iterates through getActiveStatusEffects(). - If a ShieldStatus is found, reduces the incoming damage by the shield's reduction percentage. - Calls super.takeDamage() with the calculated amount.
+ void toxic()	Sets isPoisonShot to true.
+ void growth()	Sets the inherited projectileScale to 2.0.
+ void resetBuffs()	Calls super.resetBuffs() to reset standard multipliers and sets isStunShot to false.
+ double getLaunchX()	Returns the starting X coordinate for projectiles (getX() + 50).
+ double getLaunchY()	Returns the starting Y coordinate for projectiles, centered vertically relative to the projectile's scale.
+ double getLaunchAngle()	Returns the fixed launch angle of -60 degrees.
+ boolean isPoisonShot()	Returns false (Carrot does not use poison).
+ boolean isStunShot()	Returns the value of isStunShot.

+ Image getProjectileImage()	Returns PROJECTILE_IMG.

2.1.3 package status

2.1.3.1 Class ShieldStatus extends StatusEffect

This class represents a defensive status effect that reduces the damage taken by the character when active.

2.1.3.1.1 Fields

- final double damageReduction	A constant value representing the damage reduction factor (0.6, or 60%).

2.1.3.1.2 Constructor

+ ShieldStatus(int duration)	- Calls the superclass constructor with the provided duration. - Sets the icon path

2.1.3.1.3 Methods

+ void activate(Character target)	Activates the effect.
+ double getDamageReduction()	Getter method for the damageReduction field.

2.1.3.1 Class StunStatus extends StatusEffect

This class represents a status effect that causes the target to lose their turn due to being stunned.

2.1.3.2.1 Constructor

+ StunStatus(int duration)	- Calls the superclass constructor with the provided duration. - Sets the icon path

2.1.3.2.2 Methods

+ void activate(Character target)	Activates the effect.
+ boolean shouldSkipTurn()	Returns true if duration > 1

2.1.3.1 Class ToxicStatus extends StatusEffect

This class represents a poison effect that deals damage to the target character over multiple turns.

2.1.3.3.1 Fields

- double damagePerTurn	The amount of damage applied to the target each turn.
- boolean isFresh	A flag used to prevent damage application on the very first turn the status is activated (immediate application).

2.1.3.3.2 Constructor

+ ToxicStatus(int duration, Double damagePerTurn)	- Calls the superclass constructor with the provided duration. - Sets the icon path. - Assigns damagePerTurn. - Sets isFresh to true.

2.1.3.3.3 Method

+ void activate(Character target)	- Checks if isFresh is true. If so, sets isFresh to false and returns immediately - Otherwise, calls target.takeDamage(damagePerTurn)

2.1.4 **package** unit

2.1.4.1 **Class Projectile** extends GameObject

This class represents the projectile thrown by the characters. It handles physics calculations, collision detection, damage calculation, and rendering.

2.1.4.1.1 Fields

- static final double GRAVITY	Constant defining the gravitational acceleration (1750 pixels/second^2).
- final Image image	The image representing the projectile.
- double vx, vy	Current velocity components in X and Y directions.

- final double initialVx	The initial velocity in the X direction, used for damage calculation scaling.
- boolean isDestroyed	Flag indicating if the projectile has been destroyed.
- final ProjectileStats stats	Container for projectile statistics like damage, scale, and status effects.
- final Wind wind	Reference to the wind mechanics to apply force during updates.

2.1.4.1.2 Constructor

+ Projectile(double startX, double startY, Image image, double angle, double power, Wind wind, ProjectileStats stats)	- Calls the superclass constructor super(startX, startY). - Assigns image, stats, and wind. - Size Initialization. - Sets initial position to startX, startY. - Calculates velocityPower as power * 18.0. - Calculates vx and vy using trigonometry based on the launch angle. - Stores initialVx.

2.1.4.1.3 Methods

+ Rectangle2D getHitBox()	Returns the collision hitbox depends on characters.
+ void update(double deltaTime)	Updates physics and state per frame.
+ void render(GraphicsContext gc)	Renders the projectile.
+ double getDamage()	- Calculates damage based on momentum retention. - Returns stats.getDamage() multiplied by the ratio of current vx to initialVx.
+ boolean isIntersects(GameObject other)	Returns false
Getters/Setters	

2.1.4.2 Class ProjectileStats

This class serves as an immutable data container (DTO) that holds the statistical properties of a projectile, such as damage, size, and special effects.

2.1.4.2.1 Fields

- final double damage	The base damage value of the projectile.
- final double scale	The scaling factor for the projectile's size.
- final boolean isPoisonous	Flag indicating if the projectile applies a poison effect.
- final boolean isStun	Flag indicating if the projectile applies a stun effect.
- final int poisonDuration	The duration (in turns) of the poison effect (default: 2 if poisonous).
- final double poisonDamage	The damage per turn for the poison effect (default: 10 if poisonous).

2.1.4.2.2 Constructor

+ ProjectileStats(double damage, double scale, boolean isPoisonous, boolean isStun)	- Assigns damage, scale, isPoisonous, and isStun from parameters. - If isPoisonous is true: Sets poisonDuration to 2 and poisonDamage to 10. Otherwise: Sets both poisonDuration and poisonDamage to 0.

2.1.4.2.3 Methods

Getters	

2.1.4.3 Class Wall extends GameObject

This class represents a wall entity in the game environment, primarily used for collision boundaries.

2.1.4.3.1 Constructor

+ Wall(double x, double y, double width, double height)	- Calls the superclass constructor super(x, y). - Sets the width and height of the wall.

2.1.4.3.2 Methods

+ Rectangle2D getHitBox()	Returns a Rectangle2D representing the wall's collision boundary (x, y, width, height).
+ void update(double deltaTime)	Empty implementation (walls do not currently require per-frame updates).
+ void render(GraphicsContext gc)	Renders the wall.

2.2 package util

2.2.1 Class UIButton

2.3.1.1 Method

+ static StackPane SignButton(String content, String imagePath, Runnable action)	- Creates a styled sign button and event handling.

2.3 package gui

2.3.1 Class IntroMenu extends VBox

This class represents the first introduction screen of the game. It displays the game title, background, and a start button.

2.3.1.1 Fields

- Runnable onAction	A Runnable that defines the action to be executed when the start button is triggered.

2.3.1.2 Constructor

+ introMenu()	- Initialize the Intro Menu pane (spacing 30, alignment CENTER). - Set the background image using "background/opening.png". - Initialize the header text "VVARGETABLE" with custom font "bringa.ttf", styling with stroke and drop shadow. - Create "START GAME" button with "transparent background. - Set mouse listeners for hover effects (change stroke color) and sound effects (SoundManager).

	- Add header and button to the children list.

2.2.1.3 Methods

+ void setOnAction(Runnable onAction)	Setter method for the onAction field.

2.3.2 Class **MainMenu** extends VBox

This class represents the main menu screen of the game, allowing the player to navigate to the "How to Play" section or start the game.

2.3.2.1 Fields

- Runnable onHowToAction	A Runnable that defines the action to be executed when the "How to Play" button is clicked.
- Runnable onStartAction	A Runnable that defines the action to be executed when the "Play" button is clicked.

2.3.2.2 Constructor

+ MainMenu()	- Initialize the Main Menu pane. - Set alignment to CENTER and spacing to 30. - Set the background. - Initialize the "Welcome to VVGETABLE game" text with custom font. - Create "HOW TO PLAY" and "PLAY" buttons using UIButton.SignButton. - Add text and buttons to the children list.

2.3.2.3 Methods

+ void setOnHowToAction(Runnable onHowToAction)	Setter method for the onHowToAction field.
+ void setOnStartAction(Runnable onStartAction)	Setter method for the onStartAction field.

2.3.3 Class **HowToMenu** extends VBox

This class represents the "How to Play" screen, displaying game instructions, skill descriptions, and a button to start the game.

2.3.3.1 Fields

- Runnable onStartAction	A Runnable that defines the action to be executed when the "PLAY" button is clicked.

2.3.3.2 Constructor

+ HowToMenu()	- Initialize the How-to Play menu (alignment CENTER, spacing 30). - Set the background image. - Initialize the header "HOW TO PLAY" with custom font. - Create instruction texts (line1 - line6) explaining game controls, mechanics, and cooldowns, applying specific colors (DARKCYAN, RED, BLACK). - Create a row of skill descriptions (skillRow) displaying icons and names for 6 skills using SkillDescription method. - Create a "PLAY" button using UIButton.SignButton. - Add all elements to the children list.

2.3.3.3 Methods

- VBox SkillDescription(String imagePath, String description)	- Loads the skill icon image. - Creates a text description (Tahoma, Bold, size 16, color #3E2723). - Returns a centered VBox containing the icon and text.
+ void setOnStartAction(Runnable onStartAction)	Setter method for the onStartAction field.

2.3.4 Class ModeSelectMenu extends VBox

This class represents the mode selection screen, allowing the player to choose between "Doomed" and "Classic" game modes.

2.3.4.1 Fields

- Runnable onDoomedAction	A Runnable that defines the action to be executed when the "DOOMED" mode button is clicked.
- Runnable onClassicAction	A Runnable that defines the action to be executed when the "CLASSIC" mode button is clicked.

2.3.4.2 Constructor

+ ModeSelectMenu()	- Initialize the How-to Play menu (alignment CENTER and spacing to 30) and set background. - Create "SELECT MODE" text - Create "DOOMED" button using UIButton.SignButton. Add left/right decorative icons to the button pane. - Create "CLASSIC" button using UIButton.SignButton. Add left/right decorative icons to the button pane. - Add the header text and both buttons to the children list

2.3.4.3 Methods

+ void setOnDoomedAction(Runnable onDoomedAction)	Setter method for the onDoomedAction field.
+ void setOnClassicAction(Runnable onClassicAction)	Setter method for the onClassicAction field.

2.3.5 Class GamePane extends StackPane

This class represents the main game area where all game objects are rendered. It manages the canvas and the background of the gameplay scene.

2.3.5.1 Fields

- Canvas canvas	The canvas element used for drawing game graphics.
- ArrayList<GameObject> allObjects	A list containing all the game objects that need to be rendered on the screen.
- GraphicsContext gc	The graphics context associated with the canvas, used for drawing operations.

2.3.5.2 Constructor

+ GamePane(Canvas canvas, ArrayList<GameObject> allObjects)	- Initialize the GamePane. - Assigns the canvas and allObjects fields from parameters. - Initializes gc from canvas.getGraphicsContext2D(). - Adds the canvas to this pane's children. - Set the background image with background size set to cover with 100% width/height.

2.3.5.3 Methods

+ void drawObjects()	- Clears the entire canvas. - Iterates through allObjects and calls the render(gc) method for each object to draw them on the screen.

2.3.6 Class **SkillButton** extends StackPane

This class represents a UI component for a specific skill. It handles the display of the skill icon, cooldown status, and user interaction logic.

2.3.6.1 Fields

- Skill skill	The skill object associated with this button.
- Button btn	The main button component for interaction.
- Text cooldownText	Text element displaying the remaining cooldown turns.
- ImageView iconView	Displays the skill's icon.
- boolean isOwnerTurn	Tracks whether it is currently the owner's turn to enable/disable interaction.
- GameManager gameManager	Reference to the game manager to handle skill usage events.

2.3.6.2 Constructor

+ SkillButton(Skill skill, GameManager gm)	- Initialize the SkillButton. - Assign skill and gameManager fields. - Create ImageView using skill.getIcon() with width and height set to 50. - Set style to transparent background with hand cursor. Set action handler to use the skill, update UI, and notify gameManager if the skill is ready and it is the owner's turn. Initialize text and set initial visibility to false. - Install a tooltip with the skill name. - Add the button and cooldown text to the pane's children and call updateUI().

2.3.6.3 Methods

+ void updateUI()	- If skill is ready AND it is the owner's turn: • Enable the button. • Remove any visual effects from iconView. • Hide cooldownText. Otherwise: • Disable the button. • Apply a grayscale effect (ColorAdjust saturation -1) to iconView. • If the skill is NOT ready (on cooldown), display cooldownText with the current cooldown value; otherwise, hide it.
+ void setTurnActive(boolean isActive)	Set the isOwnerTurn status and update the UI.
+ void onTurnStart()	Call skill.reduceCooldown() and set the turn as active.

2.3.7 Class CustomHpBar extends Pane

This class represents a custom health bar with smooth transition animations. It supports mirroring for players on different sides of the screen.

2.3.7.1 Fields

- Rectangle bar	Background rectangles representing the current HP.
- Rectangle bg	Foreground rectangles representing the total HP capacity.
- double maxWidth	The maximum width of the HP bar.
- Timeline smoothAnim	A timeline used to animate the HP bar changes smoothly.
-	
-	
-	Constants defining the HP bar dimensions (Width: 480, Height: 20).

2.3.7.2 Constructor

+ PlayerHpBarPane(Carrot carrot, Onion onion)	- Initialize the Custom HP Bar. - Set maxWidth to the provided width. - Create bg (width, height, DARKGRAY) with rounded corners (arc size = height). - Create bar (width, height, LIMEGREEN) with rounded corners. - Set default alignment of bar to Pos.CENTER_RIGHT (bar shrinks from left to right). - Add bg and bar to the pane's children.

2.3.7.3 Methods

+ void setHp(double percent)	Updates the HP bar width with a smooth animation.
+ void setRightSide()	Sets the alignment of the HP bar to CENTER_LEFT. This is used for the right-side player (Onion) to make the bar shrink from right to left (towards the center).

2.3.8 Class PlayerStatusPane extends Pane

This class manages the status display for both players, including their custom health bars, active skill buttons, and status effect icons.

2.3.8.1 Fields

- final CustomHpBar onionHpBar	The custom health bar for the Onion player.
- final CustomHpBar carrotHpBar	The custom health bar for the Carrot player.
- final Carrot carrot	Reference to the Carrot player unit.
- final Onion onion	Reference to the Onion player unit.
- final HBox p1SkillBox	Container for Player 1 (Carrot) skill buttons.
- final HBox p2SkillBox	Container for Player 2 (Onion) skill buttons.
- final VBox p1StatusBox	Container for Player 1 status effect icons.
- final VBox p2StatusBox	Container for Player 2 status effect icons.

2.3.8.2 Constructor

+ PlayerStatusPane(Carrot carrot, Onion onion)	- Assigns carrot and onion fields. - Create p1SkillBox and p2SkillBox - Create p1StatusBox and p2StatusBox

	- Initialize onionHpBar and carrotHpBar using CustomHpBar - Add all bars and boxes to the pane's children and set background to transparent. - Calls update() to set initial values.

2.3.8.3 Methods

+ void update()	Updates UI components.
+ void setSkills(SkillButton s1, SkillButton s2, SkillButton s3, SkillButton s4, SkillButton s5, SkillButton s6)	Sets up skill buttons.
- void updateStatusIcons(VBox statusBox, entity.base.Character character)	update status icons.

2.3.9 Class PowerBarPane extends Pane

This class represents the power charging bar that appears above the character when they are charging their attack.

2.3.9.1 Fields

- GameManager gameManager	Reference to the game manager to access turn state and power logic.
- Rectangle bg	The background rectangle of the power bar.
- Rectangle fg	The foreground rectangle that changes width to represent power level.
- final double BAR_WIDTH	Constant defining the bar width (100).
- final double BAR_HEIGHT	Constant defining the bar height (15).
- final double Y_OFFSET	Constant defining the vertical offset (20) to position the bar above the player.

2.3.9.2 Constructor

+ PowerBarPane(GameManager gameManager)	- Assigns the gameManager field. - Create bg (100x15, DARKGRAY) with a BLACK stroke. - Create fg (0x15, RED).

	- Add bg and fg to the pane's children. - Set initial visibility to false (hidden).

2.3.9.3 Methods

+ void update(Character activePlayer)	- Checks gameManager.getCurrentTurnState(). - If state is CHARGING: • Set visibility to true. • Calculate the new position to center the bar above the activePlayer. • Update setTranslateX and setTranslateY. • Calculate power percentage (currentPower / maxPower) and update the width of powerBarForeground. Otherwise: • Set visibility to false.

2.3.10 Class TurnInfoPane extends VBox

This class represents a panel that displays the countdown timer, the current player's turn, and a wind indicator showing strength and direction.

2.3.10.1 Fields

- Text turnText	Text element displaying the current player's name.
- Text timerText	Text element displaying the remaining time in seconds.
- Rectangle windBar	The rectangle that visually represents the wind's strength.
- ImageView windArrow	An image showing the direction of the wind.
- StackPane windBarContainer	A container holding the wind frame and the wind bar.
- final double MAX_WIND_BAR_WIDTH	Constant defining the maximum visual width of the wind bar (60).
- final double MAX_WIND_STRENGTH	Constant defining the maximum possible wind strength (100).
- int lastWindDirection	Tracks the previous wind direction to manage arrow flipping logic.

2.3.10.2 Constructor

+ TurnInfoPane()	- Set alignment to TOP_CENTER, spacing 5, Y-translation 20, and mouseTransparent to true. - Initialize turnText and timerText. - Create a background windFrame. - Create windBar and add it with the frame to windBarContainer. - Load arrow image, create windArrow (fit width 100), and preserve ratio. - Add all elements to the VBox children.

2.3.10.3 Methods

+ void updateUI(int seconds, PlayerTurn currentTurn, Wind wind)	Updates all UI components (Timer, Turn Text, Wind bar, Wind arrow).

2.3.11 Class GameOverPane extends StackPane

This class represents the game over screen, displaying the winner's name and providing options to retry the game or return to the menu.

2.3.11.1 Fields

- Runnable onRestart	A Runnable that defines the action to be executed when the "RETRY" button is clicked.
- Runnable onExit	A Runnable that defines the action to be executed when the "MENU" button is clicked.

2.3.11.2 Constructor

+ GameOverPane(String winnerName)	- Initialize the Game Over pane (alignment CENTER). - Set background style to semi-transparent black (rgba(0, 0, 0, 0.5)). - Create a window with background image "background/howTo.png". - Display the winner's name using "Verdana" font (Bold, size 60).

	- Color Logic: Set text color to ORANGE if winner is "CARROT", or PURPLE if winner is "ONION". - Create "RETRY" and "MENU" buttons using UIButton.SignButton. - Add title and buttons to the window content.

2.3.11.3 Methods

+ void setOnRestart(Runnable onRestart)	Setter method for the onRestart field.
+ void setOnExit(Runnable onExit)	Setter method for the onExit field.

2.4 package logic

2.4.1 package base

2.4.1.1 Class GameManager

This class acts as the central controller for the game. It coordinates between various managers, handles the main game loop, manages game states, and processes high-level game logic like turn switching and game over conditions.

2.4.1.1.1 Fields

+ boolean isGameOver	Flag indicating if the game has ended.
- StackPane root	The root node of the JavaFX scene graph.
- SceneManager sceneManager	Manages scene transitions (Intro, Game, Game Over).
- TurnManager turnManager	Manages turn states, timers, and wind mechanics.
- EntityManager entityManager	Manages game entities (Players, Projectiles, Walls).
- PowerMechanic powerMechanic	Handles the logic for charging projectile power.
- CollisionHandler collisionHandler	specific handler for detecting and resolving collisions.
- SkillManager skillManager	Manages player skills and their UI buttons.
- PlayerStatusPane playerStatusPane	UI component displaying player health and status.

- PowerBarPane powerBarPane	UI component displaying the charge power bar.
- TurnInfoPane turnInfoPane	UI component displaying turn timer and wind info.
- Canvas gameCanvas	The canvas where the game world is rendered.
- Projectile currentProjectile	Reference to the currently active projectile in the air.
- InputHandler inputHandler	Handles keyboard and mouse inputs.
- ImageCursor carrotCursor, onionCursor	Custom cursors for each player's turn.
- boolean isDoomedMode	Flag indicating if the "Doomed" game mode is active.

2.4.1.1.2 Constructor

+ GameManager(StackPane root)	- Assigns the root pane. - Initializes all managers. - Initializes gameCanvas. - Loads images to create ImageCursor objects with centered hotspots.

2.4.1.1.3 Methods

+ void start()	Calls sceneManager.showIntroMenu() to display the start screen.
+ void startGame()	- Resets turnManager and initializes entityManager. - Selects background image based on game mode. - Creates game and UI panes - Calls sceneManager.showGameScene() to switch to the gameplay view. - Initializes skillManager, input handlers, resets power, and updates the cursor. - Starts the GameLoop animation timer.
+ void launchProjectile()	- Gets the active player and creates a projectile using current power and wind.

	- Adds the projectile to entityManager. - Disables all skills and plays the throw sound.
+ void update(double deltaTime)	- Updates turnManager and UI panes. - Updates powerMechanic. - Checks collisions via collisionHandler. If a hit occurs, records hitTimestamp. Cleans up destroyed projectiles. - Waits for a duration before transitioning to CHANGING_TURN. - Calls switchTurn() and resets power.
+ void switchTurn()	- Switches the active player turn. - If Stunned: Displays a "STUNNED! (SKIP TURN)" text overlay for 1.5 seconds, then forces the turn to end. - Updates the cursor and skill buttons.
+ void checkGameOver()	Checks if either player is dead. If true, sets isGameOver to true.
+ void showGameOverWindow()	Determines the winner ("ONION" or "CARROT") and calls sceneManager.showGameOverWindow().
+ void updateCursor()	Sets the scene cursor to carrotCursor or onionCursor based on the current player's turn.
Getters/Setters	

2.4.2 Class **GameLoop** extends AnimationTime

This class manages the main game loop, handling the timing of updates, physics calculations, and rendering frame-by-frame..

2.4.2.1 Fields

- final GameManager gameManager	Reference to the main game controller.
- final GamePane gamePane	Reference to the game pane for rendering objects.

- final PlayerStatusPane playerStatusPane	Reference to the status pane to update UI elements.
- final ArrayList<GameObject> allObjects	List of all game objects that need to be updated.
- long lastNanoTime	Stores the timestamp of the previous frame to calculate deltaTime.

2.4.2.2 Constructor

+ GameLoop(GameManager gameManager, GamePane gamePane, PlayerStatusPane playerStatusPane, ArrayList<GameObject> allObjects)	Initializes the loop by assigning all the provided manager and pane references to their respective fields.

2.4.2.3 Methods

+ void handle(long currentNanoTime)	- If gameManager.isGameOver is true, stops the timer, prints "Game Over!", and calls gameManager.showGameOverWindow(). - Calculates time elapsed since the last frame in seconds. - Iterates through allObjects and calls update(deltaTime) for each. - Calls gameManager.update(deltaTime). - Calls playerStatusPane.update(). - Calls gamePane.drawObjects() to redraw the scene. - Calls gameManager.checkGameOver() to see if the game has ended this frame.

2.4.3 package logic.enums

2.4.3.1 Enum PlayerTurn

Enum Constant	Description
PLAYER_ONE	Represents the first player's turn (Carrot).
PLAYER_TWO	Represents the second player's turn (Onion).

2.4.3.1 Enum Turnstate

Enum Constant	Description
READY	The state when the player is ready to take action.
CHARGING	The state when the player is holding the mouse to charge power.
PROJECTILE_IN_AIR	The state when a projectile has been launched and is moving.
CHANGING_TURN	The state during the transition between player turns.
HIT	The state triggered immediately after a projectile hits a target.

2.4.4 package logic.handlers

2.4.4.1 Class CollisionHandler

2.4.4.1.1 Fields

- TurnManager turnManager	Reference to the Turn Manager to update the turn state

2.4.4.1.2 Constructor

+ CollisionHandler(TurnManager turnManager)	Initialize Turn Manage

2.4.4.1.3 Methods

+ long handleCollision(Projectile projectile, Character opponent, Wall wall)	Checks for intersections between the projectile and the opponent or the wall. - Applies damage and status effects if the opponent is hit. - Destroys the projectile upon collision. - Update TurnState. - Return timestamp of the collision if a hit occurs otherwise returns 0.

2.4.4.2 Class InputHandler

2.4.4.2.1 Fields

- final GameManager gameManager	Reference to the main game manager.

2.4.4.2.2 Constructor

+ InputHandler(GameManager gameManager)	- Initializes fields using parameters

2.4.4.2.3 Methods

+ void attachEventHandlers(Scene scene)	- Sets up mouse event listeners (pressed and released) on the game scene.
+ void handleMousePressed(MouseButton button, double x, double y)	- Handles logic when the mouse is pressed. Starts the charging state if the player clicks on their character when the state is READY.
+ void handleMouseReleased(MouseButton button)	- Handles logic when the mouse is released. Launches the projectile if the state is CHARGING state.

2.4.5 package logic.managers

2.4.5.1 Class TurnManager

2.4.5.1.1 Fields

- PlayerTurn currentPlayerTurn	Tracks player turn.
- TurnState currentTurnState	Tracks currenstate of turn
- TurnTimer turnTimer	Manages the countdown timer for turn
- Wind wind	Manages the wind to affect projectiles

2.4.5.1.2 Constructor

+ TurnManager()	- Initializes the manager with Player One's turn, READY state, a 10-second timer, and a new Wind instance.

	- Plays the ticking sound when time is 5 seconds or less.

2.4.5.1.3 Methods

+ void update(double deltaTime)	- Updates the turn timer during READY or CHARGING states. Changes state to CHANGING_TURN if time runs out.
+ void switchTurn()	- Switches the current player, updates wind for the new round, resets the timer, and sets state to READY.
+ void reset()	- Resets manager to the initial state
+ void forceEndTurn()	- Forces state to CHANGING_TURN
+ Getters	

2.4.5.2 Class EntityManager

2.4.5.2.1 Fields

- Carrot carrot	The Carrot player character.
- Onion onion	The Onion player character.
- Wall wall	The wall object
- final ArrayList<GameObject> allObjects	A list containing all game objects.

2.4.5.2.2 Constructor

+ EntityManager()	- Initializes the ArrayList of game objects.

2.4.5.2.3 Methods

+ void initializeGameWorld()	- Creates and positions the Carrot, Onion, and Wall objects, and adds to the allObjects list.
+ void addProjectile(Projectile p)	- Adds a projectile to the game world.
+ void cleanupDestroyed(Projectile currentProjectile)	- Removes a destroyed projectile from object list.
+ Getters	

2.4.5.3 Class SkillManager

2.4.5.3.1 Fields

+ final GameManager gameManager	Reference to the GameManager.
+ final PlayerStatusPane playerStatusPane	Reference to the UI pane displaying player status and skills.
- private SkillButton p1Skill1...p2Skill3	SkillButton for Player 1 and Player 2

2.4.5.3.2 Constructor

+ SkillManager(GameManager gameManager, PlayerStatusPane playerStatusPane)	- Initializes fields using parameters

2.4.5.3.3 Methods

+ void initializeSkills()	- Creates all skill instances and assigns them to SkillButtons. - Sets initial active states of SkillButton to true for Player 1 and false for Player2.
+ void disableAllSkills()	- Deactivates all skill buttons when a projectile is fired.
+ void handleSkillUsage(PlayerTurn currentTurn)	- Disables the skills of the current player after a skill is used.

2.4.5.4 Class SceneManager

2.4.5.4.1 Fields

- final StackPane root	Reference to the GameManager.
- final GameManager gameManager	Reference to the UI pane displaying player status and skills.
- IntroMenu introMenu	The introduction menu screen.
- MainMenu mainMenu	The main menu screen.
- HowToMenu howToMenu	The instructions screen.
- ImageCursor carrotCursor	The custom cursor for Carrot's turn.
- ImageCursor onionCursor	The custom cursor for Onion's turn.

2.4.5.4.2 Constructor

+ SceneManager(StackPane root, GameManager gameManager)	- Initializes the scene manager, menus, and cursors.

2.4.5.4.3 Methods

- void initializeMenus()	- Creates all skill instances and assigns them to SkillButtons. - Sets initial active states of SkillButton to true for Player 1 and false for Player2.
- void initializeCursors()	- Deactivates all skill buttons when a projectile is fired.
+ void showIntroMenu()	- Displays the Intro Menu.
- void showMainMenu()	- Displays the Main Menu.
- void showHowToMenu()	- Displays the How-To Menu.
+ void showGameScene(GamePane gamePane, PlayerStatusPane playerStatusPane, PowerBarPane powerBarPane, TurnInfoPane turnInfoPane)	- Switches the scene to the main game view with game pane, status pane, power bar, and turn info.
+ void showGameOverWindow(String winner)	- Displays the Game Over window with the winner's name.
+ void updateCursor(PlayerTurn turn)	- Updates the mouse cursor depends on the current player's turn.

2.4.5.5 Class SoundManager

2.4.5.5.1 Fields

- final AudioClip throwingSound	The throwing sound.
- final AudioClip strikeSound	The sound played when player take damage.
- final AudioClip skillUsedSound	The activate skill sound.
- final AudioClip tickingSound	The sound played when timer is running out(last 5 seconds).
- final AudioClip clickSound	The button click sound.

- final AudioClip hoverSound	The button hover sound.
- final MediaPlayer bgmPlayer	The Background music.

2.4.5.5.2 Static Initialization

	- Initializes fields ,sets volume and loops cycle.

2.4.5.5.3 Methods

+ void playThrowSound()	- Plays the throwing sound effect.
+ void playStrikeSound()	- Plays the strike/damage sound effect.
+ void playSkillSound()	- Plays the skill activation sound effect.
+ void playTickingSound()	- Plays the ticking sound effect
+ void playClickSound()	- Plays the button click sound effect.
+ void playHoverSound()	- Plays the button hover sound effect.
+ void playBackgroundMusic()	- Plays the background music in a loop.

2.4.6 package logic.mechanics

2.4.6.1 Class Wind

2.4.6.1.1 Fields

- double strength	The current strength of the wind
- final double MAX_STRENGTH	The maximum possible magnitude of the wind strength (set to 100).
- Random random	Random number generator to set wind strength and duration.
- int duration	The number of turns before the wind changes.

2.4.6.1.2 Constructor

+ Wind()	- Initializes the random filed and call randomize().

2.4.5.5.3 Methods

+ void onNewRound()	- Updates the wind status at the start of a new round. Decrements the duration and call randomize() if duration reaches zero.
- void randomize()	- Generates a random strength between -MAX_STRENGTH and +MAX_STRENGTH and sets a random duration (3 to 5 rounds).
+ int getDirection()	- Returns the direction of the wind (1 for right, -1 for left, 0 for no wind).
+ double getStrengthMagnitude()	- Returns the absolute value of the wind strength.
+ Getters	

2.4.6.2 Class TurnTimer

2.4.6.2.1 Fields

- double maxTime	The maximum time allowed for a turn in seconds.
- double currentTime	The current remaining time for the turn.
- boolean isTimeOut	A flag indicating if the time has run out.

2.4.6.2.2 Constructor

+ TurnTimer(double maxTimeSeconds)	-Initializes the timer with a specified maximum time and call reset().

2.4.6.2.3 Methods

+ void tick(double deltaTime)	- Decreases the currentTime by deltaTime. Sets isTimeOut to true if time reaches zero.
+ void reset()	- Resets the timer to the maximum time and clears the timeout flag.
+ int getCurrentTimeInt()	- Returns the current time rounded up to the nearest integer.
+ double getStrengthMagnitude()	- Returns the absolute value of the wind strength.

+ Getters	

2.4.6.3 Class PowerMechanic

2.4.6.3.1 Fields

# String name	The name of the skill.
# Image icon	The icon image of the skill.
# int maxCooldown	The number of turns required to recharge the skill.
# int currentCooldown	The current remaining cooldown turns.
# boolean isActive	Indicates if the skill is currently active.

2.4.6.3.2 Constructor

+ Skill(String name, String iconPath, int cooldown)	- Initializes the skill with name, icon, and cooldown.

2.4.6.3.3 Methods

+ void reset()	- Resets currentPower to MIN_POWER.
+ void charge(double deltaTime)	- Increases currentPower based on POWER_CHARGE_RATE and deltaTime, capped at MAX_POWER.
+ Getters	

2.4.7 package logic.skills

2.4.6.1 Abstract Class Skill

2.4.7.3.1 Fields

# String name	The name of the skill.
# Image icon	The current remaining time for the turn.
# int maxCooldown	The number of turns required to recharge the skill.
# int currentCooldown	The current remaining cooldown turns.
# boolean isActive	If the skill is currently active or not.

2.4.7.3.2 Constructor

+ Skill(String name, String iconPath, int cooldown)	- Initializes the skill with name, icon, and cooldown.

2.4.7.3.3 Methods

+ *void activate (GameManager gameManager)*	Abstract method to define the specific effect of the skill.
+ void reduceCooldown()	Decreases the cooldown by 1 if it is greater than 0.
+ boolean isReady()	Checks if the skill is ready to be used (cooldown <= 0).
+ Getters	

2.4.6.2 **Class Double** extends Skill

2.4.7.3.1 Constructor

+ Double()	- Initializes the "DOUBLE" skill with a specific icon and 5-turn cooldown.

2.4.7.3.2 Methods

+ *void activate (GameManager gameManager)*	-Activates the doubleDamage effect on the active player.

2.4.6.3 **Class Heal** extends Skill

2.4.7.3.1 Constructor

+ Heal()	- Initializes the "HEAL" skill with a specific icon and 5-turn cooldown.

2.4.7.3.2 Methods

+ *void activate (GameManager gameManager)*	-Heals the active player for 30 HP.

2.4.6.4 **Class Stun** extends Skill

2.4.7.3.1 Constructor

+ Stun()	- Initializes the "STUN" skill with a specific icon and 5-turn cooldown.

2.4.7.3.2 Methods

+ *void activate (GameManager gameManager)*	-Activates the stunShot effect on the active player.

2.4.6.5 Class **Toxic** extends Skill

2.4.7.3.1 Constructor

+ Stun()	- Initializes the "STUN" skill with a specific icon and 5-turn cooldown.

2.4.7.3.2 Methods

+ *void activate (GameManager gameManager)*	-Activates the stunShot effect on the active player.

2.4.6.6 Class **Growth** extends Skill

2.4.7.3.1 Constructor

+ Growth()	- Initializes the "GROWTH" skill with a specific icon and 5-turn cooldown.

2.4.7.3.2 Methods

+ *void activate (GameManager gameManager)*	-Activates the growth effect (increase projectile size) on the active player.

2.4.6.7 Class **Shield** extends Skill

2.4.7.3.1 Constructor

+ Shield()	- Initializes the "SHIELD" skill with a specific icon and 5-turn cooldown.

2.4.7.3.2 Methods

+ *void activate (GameManager gameManager)*	-Applies a ShieldStatus effect (damage reduction) to the active player for 2 turns.

2.5 package application

2.5.1 Class Main extends Application

2.5.1.1 Methods

+ void start(Stage primaryStage)	- Creates a StackPane as the root node. - Creates a Scene with dimensions 1280x720. - Initializes a GameManager with the root pane and calls gameManager.start(). - Starts background music via SoundManager.playBackgroundMusic(). - Sets the stage's scene, sets the title to "vvargetable", and shows the primaryStage.
+ static void main(String[] args)	Calls launch(args) to start.